

Tools for Numerical Computing

Brian Howell

INDEFINITELY IN PROGRESS

October 4, 2025

Introduction:

I had the fortunate opportunity to pursue my graduate studies at UC Berkeley under the direction of Tarek Zohdi. Throughout my time, I was exposed to a handful of useful tools from applied mathematics, machine learning, optimization, and high performance computing. This document is an attempt to transcribe and compile my notes from a 5-year period into a reference guide for myself.

Disclaimer:

The intended audience was myself. That being said, I have shared sections of this document with colleagues and friends who claimed it was helpful in their studies. Maybe it will be helpful to you.

This is a live document that I intend to continue to update, which means there are likely to be some errors. If you find mistakes and feel like telling me, I will be grateful and happy to hear from you, even for the most trivial of errors. You can reach me by email at bhowell@berkeley.edu.

Contents

Section 1: AI: Large-Language Models	1
2 The Large Language Model Pipeline	1
1.1 Stage 1: Dataset for Pretraining	1
1.2 Stage 2: Tokenization	2
1.3 Stage 3: Neural Network Training	3
1.4 Stage 4: Inference	3
1.5 Stage 5: Post Training	3
1.6 Architecture	3
3 Transformers	3
1.1 Word and Positional Embeddings	3
1.2 The Positional Update	4
1.3 Self-Attention Mechanism	4
1.4 Multi-Head Attention	5
1.5 Position-wise Feedforward Network	6
1.6 Residual Connections and Layer Normalization	7

Section 1: AI: Large-Language Models

2 The Large Language Model Pipeline

The following section is a compilation of my notes over the years about LLMs and transformer architecture, much of which is based on miscellaneous papers and lessons provided by Andrej Karpathy.

Large-Language Models have been surprisingly interesting from many vantage points. Initially, I had strong biases against the idea of universal function approximators, but as I have gone deep in the topic over the past couple years, I have come around around to effectiveness of these large models.

My initial biases against deep learning were primarily fueled by my background in traditional engineering. For over a century, physicists have built foundational models from first-principles, rigorously proving the underlying mathematics and empirically validating through physical experiments. The resulting product is a framework for analyzing the world with a set hierarchically compressed models, efficiently parameterized with falsifiable tests. Deep learning on the other hand appeared to build a model using a over parameterized universal function approximator, and using sad first-order optimization algorithms to rediscover models that the greatest minds already spent a century discovering.

Over years, the usefulness of deep learning has been undeniable. My initial thoughts on over-parameterization were overwritten with realizations of generalizability. My former disgust in brute-force gradient descent has been replaced with resulting models that handle higher degrees of complexity.

I first started playing around with DALL-E 2 mid 2022, and was floored by a link sent by my friend to an introduction to my own dissertation not written by me, but by ChatGPT3. Since then, I have oscillated between skepticism in the obvious lack of intelligence and the shock from the astounding coding feats that these LLMs have provided in my academic and professional work. The pivotable moment was when our team required mundane front-end work, and a state-of-the-art model was able one shot a front-facing GUI containing 2500 lines of code.

Beyond the training, the deployment of LLMs has also caught my attention. The trillions of operations required for inference boil down to the simplest of mathematical operations, and the most critical being matrix multiplication. The scale of these matrix multiply is massive and imbalance. As model sizes range from 30b-1t in size, the deployment can range from local operations on your Apple Silicon shared memory (I am currently running 30b parameters locally on my 48GB unified share memory M4 Pro), to a fully AWS/GCP node equipped with 8 H100s with a total 640GB of VRAM. In the distributed case, this of course comes with the added complexity of distributing the inference/training on all of these GPUs and managing the communication.

Then in February 2025, the DeepSeek R1 paper launched. I remember being not very impressed with their RL approach, but their paper did spark enough interest for me to crack open their V3 Base Model Technical Report. This paper blew my mind. At the time, I had just come out of a mixed-precision computing project at work that left me in awe of the engineering feats pulled in the V3 report.

Over the last couple of years, I have done some deep dives into the topic to ensure that I am on top of the modern techniques. I typically keep my notes physical notebooks, and then eventually compile them in this PDF for future reference. The following section are my notes over the years on the topic of LLMs.

1.1 Stage 1: Dataset for Pretraining

Over the last couple of decades, there has been a large open source effort by an organization called “Common Crawl” with the purpose of scouring the Internet to build a free, open repository of the world’s textual knowledge that can be used by anyone. For an lab developing their own models, this is a common starting point. Every couple of months, a new “crawl” will be released containing roughly 200-400 TiB textual content. These crawls start with a few seed pages and these seed pages follow the links and span the Internet.

However, this will not be enough, particularly since the results of the common crawl are quite rough. Organizations using Common Crawl as a starting point will need to implement their own methods of crawling, as well as filtering the text to ensure a quality dataset for training the LLM. Filtering includes removing content that may contain commonly blacklisted content (see example [University of Toulouse Capital’s Blacklist](#)).

The 🍷 FineWeb recipe

In the next subsections we will explain each of the steps taken to produce the FineWeb dataset.

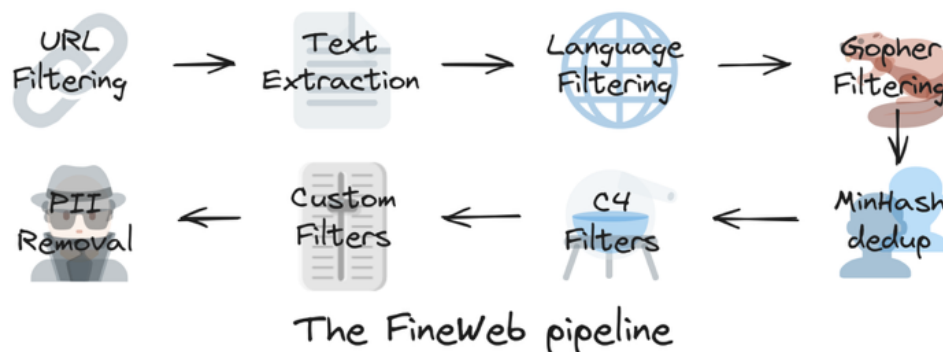


Figure 1: The Hugging Face process for filtering Common Crawl.

1.2 Stage 2: Tokenization

Once a refined representation of the Internet has been made, the next step is to represent this text in some sort of way for a neural network to process. Since the inputs to a neural network need to be numerical, we need some way to represent the text in terms of numerical values.

One way we can do this is by taking a byte, or 8 bits to sequentially represent different characters in an alphabet. With 8 bits, we get $2^8 = 256$ possible representations as numerical values of 0-255. One particular model would assign each character from the Latin-script alphabet a value. Including other common punctuation, such as space, period, etc, one could follow the patterns '0=space', '1=a', '2=b', etc.

In reality, there are several official formats that have evolved over the years. ASCII is a 7-bit encoding that enables the representation of 128 characters, and ISO 8859-1 uses 8-bit encoding. However, to extend this process across multiple languages and the infinitely expanding base of emojis, the UTF-8 encoding scheme uses 1-4 bytes to represent Unicode points. Each byte has a prefix to indicate its role:

- 1-byte characters (ASCII): '0xxxxxxx'
- 2-byte: '110xxxxx' '10xxxxxx'
- 3-byte: '1110xxxx' '10xxxxxx' '10xxxxxx'
- 4-byte: '11110xxx' '10xxxxxx' '10xxxxxx' '10xxxxxx'
- continuation byte

For example, an ASCII character is represented as a single byte: 01100001, or 97. A non-ASCII character 'é' is represented by the two bytes: '11000011' '10101001', or [195, 169]

The prefix bits in each byte tell you whether this is a single-byte character, if multi-byte is this a first or continuation byte. This format enables UTF-8 to encode over a million characters.

This means that the raw text can be viewed in several ways. It can be viewed at the bit level, the byte level, and the UTF-8 text interpretation level. Tokenization occurs at the byte level, meaning that values of 0-255 are the initial building blocks:

```
124 86 105, 101, 119 105 110 103 3283 105 110 103 108 101 32 80 111 115 116 32
```

Byte-Pair Encoding (BPE) Algorithm

- GPT3: 50,257 tokens
- GPT4: 100,277 tokens

However, once we will see in my notes on the transformer architecture, the length of the sequence being passed into is a precious commodity, as well as the numerical range (e.g., 0-255 or 0-1e5). To increase the numerical range, new tokens can be “minted” out of combinations of common tokens that appear. For example, the byte pair ‘116 32’ may occur often enough to justify the new token value of 256. The next most common byte pair could be assigned 257, and so on. It is unclear (to me) if this trade-off between numerical representation and sequence length is convex, but it appears that number of tokens (or numerical range) has stabilized out at 100,277 for GPT4. If someone knows if it is still increasing then please let me know.

1.3 Stage 3: Neural Network Training

1.4 Stage 4: Inference

1.5 Stage 5: Post Training

1.6 Architecture

3 Transformers

The transformer sought to solve fundamental issues with previous natural language approaches, specifically on how words or tokens relate to each other. “Attention is All You Need” solved this by proposing mechanism that builds an attention matrix between each of the tokens in a sequence, providing a learned similarity score based on projected embeddings. The mechanism is baked into a larger neural network architecture that has proved to be scalable and generalizable. My notes begin with the entry point of this architecture.

1.1 Word and Positional Embeddings

Neural networks do not deal with single scalar integers very well. To capture deep possible meaning behind a token, each scalar mapping between 0-100,276 for GPT4 a higher dimensional vector space better suited for deep learning. The vector is referred to as the token’s embedding (or colloquially word embedding), which can be learned during the training process.

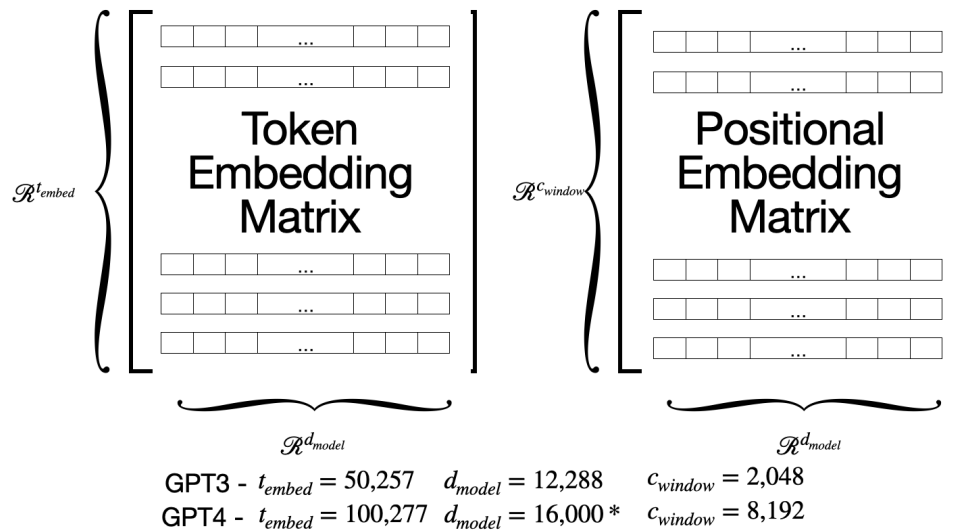


Figure 2: The embedding matrix dimensions.

No matrix operations occurs at this point, rather the token’s integer value is used to index into the embedding matrix to pluck out a row.

Why exactly is this high dimensional vector representation and why is it useful? Well, (algorithmic information content) of a scalar integer representation is limited to a token’s literal representation, which compresses any semantic or contextual meaning out. How do we quantify semantic or contextual meaning? We don’t necessarily have a first-principles approach but we do have a large corpus of material that can serve as empirical references for top-down analysis. In other words, we can represent a token or word as a higher dimension object, and “learn” the parameters from corpus. By systematically experimenting with training algorithms, our hyperparameter dimension size can also be tuned to insure that the algorithmic content of the vector is sufficiently compressed while still being sufficiently expressive. For LLMs, these dimension has been

set as $\mathbf{t} \in \mathcal{R}_{model}^d$, where $d_{model} = 12,288$ for GPT3 and $d_{model} = 16,000$ for GPT4. A great blog post by Hugging Face describes the techniques in learning these matrices under different applications: [Primer on embeddings](#). These techniques extend beyond natural language processing and also exist for applications in image, video, sound and the physical world.

Although the efficiency of the algorithmic content of a vector remains to be proven, the semantic meanings behind these embeddings provide a vector space that relate tokens of categorically similar meanings closer to one another. The vectors learned in embedding matrix are considered static, but still provide quite a bit of information about the token. 3 Blue 1 Brown has an excellent [visualization of vector embeddings](#) describing how vectors representing words such as queen and king have similar vectorial differences as the differences between other gender specific words.

However, these token representation need not be static. In fact the transformer architecture’s purpose is to process information about the sequence and update each tokens embedding based off of this information. As each embedding flows through the network, the vector is updated to a more contextually rich representation of the token in the provided sequence.

1.2 The Positional Update

The first embedding transformation incorporates a token’s position in the sequence. The way this is done has varied since the original transformer paper, but as of today it is common to directly “learn” this matrix. The dimensions of this matrix are slightly different than the token embedding matrix. The number of rows corresponds to the maximum sequence length (context window), while the number of columns matches the token embedding dimension (d_{model}) so that addition is valid. The original GPT3 model had a context window of 2,048 tokens, which is roughly 1,500 words. The original GPT4 model bumped that size up to 8,192 tokens, with subsequent models substantially bumping those numbers. Gemini 2.5 Pro (as of 2025) has a gargantuan context window size of 1 million tokens. It seems a bit silly to go into details on the context windows of any models today since these are being updated so fast that it is difficult to keep up.

For each token, its position in the sequence determines which row of the positional embedding matrix is selected. This vector is then added to the token embedding, similar to how token embeddings are selected. Visually, this is represented in Figure 3.

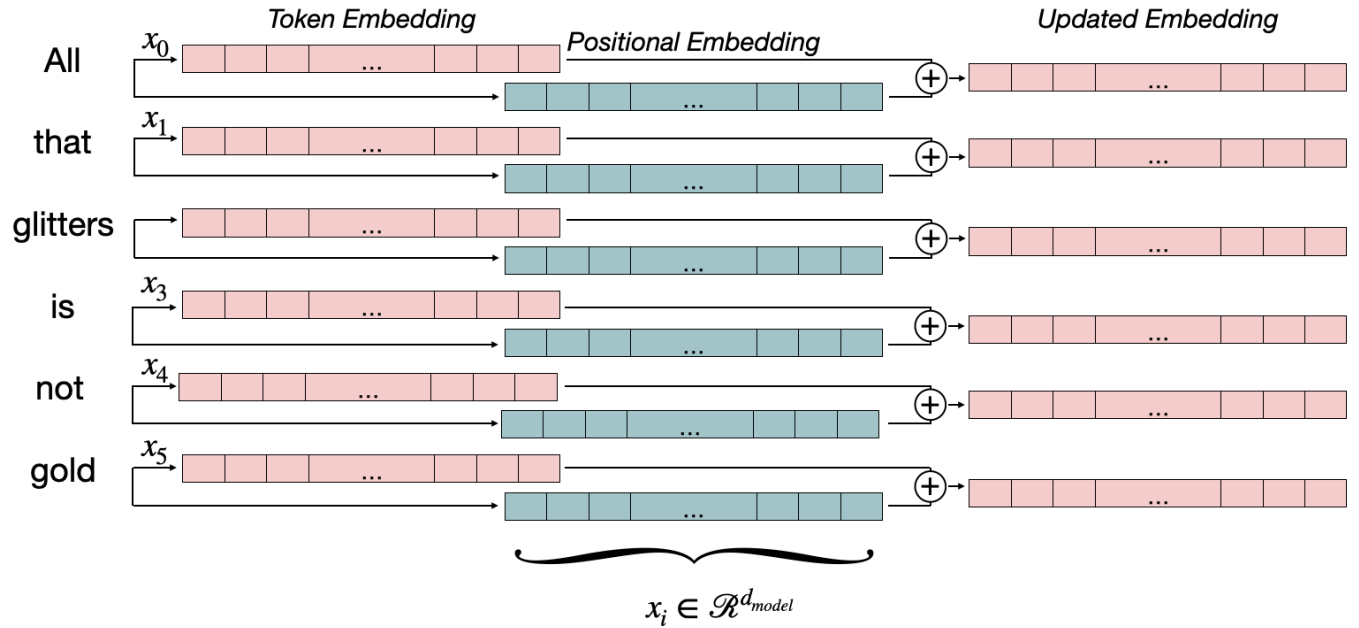


Figure 3: A token’s embedding vector is “plucked” from the token embedding matrix, and its corresponding positional vector is also “plucked” from the positional embedding matrix. These two vectors are added together and are the first update in the transformer architecture that adds context to token embeddings vectorial representation.

The output embedding transformation is a matrix of size $\mathcal{R}^{s_{length} \times d_{model}}$. One way to think about each row vector in this matrix is that each token embedding has been vectorially updated with new context information about the position. As with token embedding matrix, the weights of the positional embedding matrix are learned and are constant during inference.

1.3 Self-Attention Mechanism

The embedding matrix $X \in \mathcal{R}^{s_{length} \times d_{model}}$ is linearly projected in three different ways: Keys $\mathbf{K} = XW_K^\top$, Queries $\mathbf{Q} = XW_Q^\top$, and Values $\mathbf{V} = XW_V^\top$. I define these projections with transposes so that the output matrices preserve the same row-wise

structure as X — pedagogically convenient when following along with Figure 4. This maps embeddings from $\mathbb{R}^{s_{\text{length}} \times d_{\text{model}}}$ to a lower dimensional space $\mathbb{R}^{s_{\text{length}} \times d_{\text{head}}}$ (more on this in the section on multi-head attention).

The matrices $\mathbf{K}$, $\mathbf{Q}$, and $\mathbf{V}$ are often semantically described with some notion of “mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors.” While the original transformer paper offered this as one possible interpretation (I see no problem with this), subsequent academicians, bloggers, and influencers have continued to theory-wash the transformer architecture with database metaphors (in reality I respect most of these individuals).

Mechanistically, the attention score matrix $\mathbf{QK}^\top$ is simply the collection of pairwise inner products between two learned projections of the same input. The apparent asymmetry is not a profound statement about semantics — it arises only because the model is granted two separate projection matrices, giving it extra degrees of freedom to sculpt similarity patterns in representation space. Swap $\mathbf{Q}$ and $\mathbf{K}$ and you just get the transpose; no secret meaning revealed.

$$\sigma\left(\frac{\mathbf{QK}^\top}{\sqrt{d_{\text{head}}}}\right)$$

In reality, attention learns a bilinear similarity kernel: $\mathbf{Q}$ and $\mathbf{K}$ define the kernel, and $\mathbf{V}$ carries the information that gets reweighted once similarities are computed. The requirement that $\mathbf{Q} \neq \mathbf{K}$ is therefore functional, not conceptual. The “query/key” metaphor obscures this underlying mathematical reality and makes the mechanism seem more mysterious than it is. Sure, you can map these high-dimensional vectors onto intuitive ideas like “queries” and “keys,” but until there’s evidence that the network truly internalizes such semantics, I’ll continue yell at what I perceive as clouds passing in latent space.

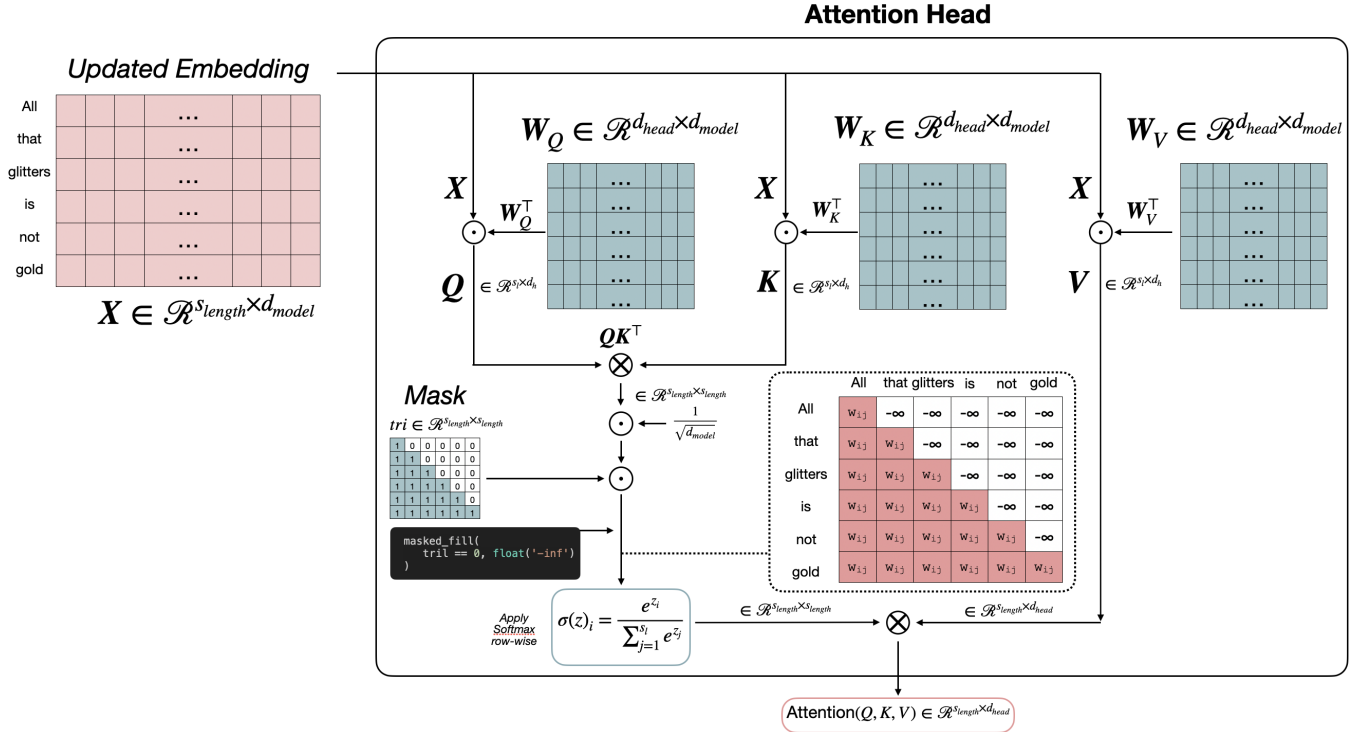


Figure 4: A single attention head.

1.4 Multi-Head Attention

A single head of attention naturally emphasizes a small subset of highly correlated pairs due to softmax’s exponential weighting by sharpening the distribution, so a single head can collapse to a dominant pattern. Multi-head attention solves this by running h independent attention heads in parallel, each with its own learned projection matrices $\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)}, \mathbf{W}_V^{(i)}$ for $i = 1, \dots, h$. Each head of attention is then capable of emphasizing possible candidate correlated pairs within subset of the embedding space.

Computationally, each head produces an output $A_{d_i} \in \mathbb{R}^{s_{\text{length}} \times d_{\text{head}}}$, where $d_{\text{head}} = d_{\text{model}}/h$. These outputs are then concatenated along the feature dimension to form $\mathbf{A} \in \mathbb{R}^{s_{\text{length}} \times d_{\text{model}}}$. A final learned projection $\mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ then mixes these head-specific insights back into a single coherent embedding. Multiple heads allow the transformer to capture diverse relationships simultaneously: local dependencies, long-range interactions, syntactic patterns, or just whatever random correlations happen to exist in the training data.

Figure 5 illustrates this process: multiple attention heads operate independently on the same input embeddings X , producing head-specific outputs A_{d_i} that are then stitched together to form the final updated embedding matrix A . Each head brings its own quirks and biases to the table, which the model can exploit—or ignore entirely.

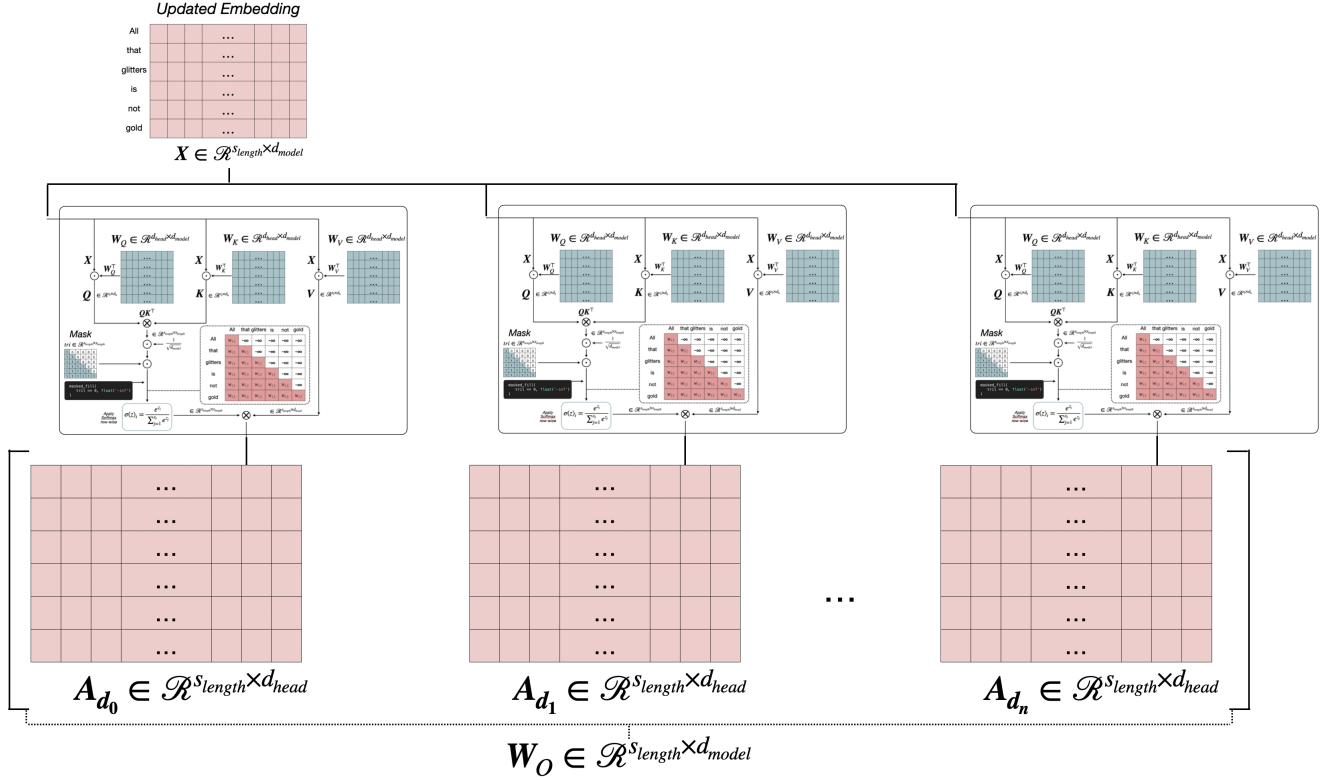


Figure 5: Multiple heads of attention.

At this point, the transformer has enriched token embeddings with contextual information by passing them through layers of self-attention. However, attention alone is not enough: it mixes information across tokens, but it does not transform that information in richer, nonlinear ways. To build depth and expressive power, the transformer architecture alternates attention layers with position-wise feedforward networks, all stabilized with residual connections and normalization. This is where the rest of the architecture comes in.

1.5 Position-wise Feedforward Network

Self-attention enriches token embeddings with contextual information, but by itself it is not a universal function approximator. The inner products and softmax are essentially linear similarity operations with a nonlinearity for weighting. To enrich the representational capacity of each token, the transformer alternates attention layers with a position-wise feedforward network (FFN).

The FFN is applied independently to each token embedding, without mixing across the sequence dimension. For a given input token embedding $x \in \mathbb{R}^{d_{model}}$, the block is defined as:

$$\text{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

where $W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$, and d_{ff} is typically larger than d_{model} (for GPT-3, $d_{ff} \approx 4 \times d_{model}$). The activation function σ was chosen as ReLU in the original transformer, but variants such as GELU and SwiGLU have since become standard in large language models. This feedforward block provides depth and nonlinear expressivity at the level of individual tokens.

Because the FFN operates identically and independently on each position, it is sometimes called a **position-wise feedforward network**. This design keeps computation efficient: the same learned transformation is applied independently and in parallel across all tokens.

Figure ?? shows the structure of the FFN block in relation to the attention layer. Together, the alternation between attention and feedforward forms the basic building block of the transformer, enabling models to scale effectively in depth and capacity.

1.6 Residual Connections and Layer Normalization

Each attention and feedforward block in the transformer is wrapped in a residual connection followed by a normalization step. These two design choices are not glamorous, but they are essential: without them, the architecture would be unstable and impossible to scale.

The residual connection adds the input of a block back to its output:

$$y = x + \text{Block}(x)$$

This bypass path ensures that the gradient can flow directly through the network, mitigating the vanishing/exploding gradient problem that plagues deep architectures. In practice, residuals make it possible to stack dozens or hundreds of transformer layers without optimization failure.

After the residual addition, the combined vector is normalized using **layer normalization**:

$$\text{LayerNorm}(y) = \frac{y - \mu}{\sigma} \cdot \gamma + \beta$$

where μ and σ are the mean and standard deviation of the features in y , and γ, β are learnable scale and shift parameters. Unlike batch normalization, which normalizes across examples in a batch, layer normalization operates along the feature dimension of a single token embedding. This makes it naturally compatible with sequence models, where batch statistics may vary widely.

The original transformer paper applied normalization *after* the residual addition (known as **post-norm**). Many modern architectures flip the order, applying normalization *before* the block (**pre-norm**), which empirically improves training stability for very deep models. The distinction is subtle but has large implications for scaling.

Figure ?? illustrates the full transformer block: a self-attention module and a feedforward network, each wrapped in residual connections and normalization. This design makes the block composable: attention mixes information across tokens, the feedforward expands nonlinear representation within tokens, and the residual-normalization wrapper stabilizes everything so the model can be trained at scale.